

```
# import needed libraries
import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import imageio.v2 as imageio
import json
from collections import defaultdict
import shutil
import cv2
from matplotlib import pyplot as plt
import torch
from torch import nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from PIL import Image
from torch.utils.data import DataLoader
from torch.utils.data import Dataset
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
device
```

```
↳ device(type='cuda')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
↳ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
```

```
# load the metadata from dataset.csv
```

```
metadata = pd.read_csv('/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/dataset.csv')
print("Shape of the dataset", metadata.shape)
# prints in a table format the first 10 entries of the dataset.csv dataset
metadata.head(10)
```

```
↳ Shape of the dataset (4083, 13)
```

	image_id	hand	leg	hip	shoulder	mixed	hardware	multiscan	fractured	fracture_count	frontal	lateral	oblique	
0	IMG0000000.jpg	0	1	0	0	0	0	1	0	0	1	1	0	
1	IMG0000001.jpg	0	1	0	0	0	0	1	0	0	1	1	0	
2	IMG0000002.jpg	0	1	0	0	0	0	1	0	0	1	1	0	
3	IMG0000003.jpg	0	1	0	0	0	0	1	0	0	0	1	1	
4	IMG0000004.jpg	0	1	0	0	0	0	1	0	0	0	1	1	
5	IMG0000005.jpg	0	1	0	0	0	0	1	0	0	0	1	1	
6	IMG0000006.jpg	1	0	0	0	0	0	1	0	0	0	1	1	
7	IMG0000007.jpg	1	0	0	0	0	0	0	0	0	0	0	1	
8	IMG0000008.jpg	0	1	0	0	0	0	1	0	0	0	1	1	
9	IMG0000009.jpg	0	1	0	0	0	0	0	0	0	0	0	1	

Next steps: [Generate code with metadata](#) [View recommended plots](#) [New interactive sheet](#)

```
# check image type
img = cv2.imread('/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/images/Fractured/IMG0000019.jpg')
print(img.shape)
```

```
↳ (2880, 2304, 3)
```

```
# median frequency balancing between all images (fractured and non-fractured)
```

```
label = [ 'hand', 'leg', 'hip', 'shoulder', 'mixed']
```

```
def estimate_weights_mfb(label):
    class_weights = np.zeros_like(label, dtype=float)
    counts = np.zeros_like(label)
```

```

for i,l in enumerate(label):
    counts[i] = metadata[metadata[l] == 1].shape[0]
counts = counts.astype(float)
median_freq = np.median(counts)
for i, label in enumerate(label):
    class_weights[i] = median_freq / counts[i]
return class_weights

classweight= estimate_weights_mfb(label)

for i in range(len(label)):
    print(label[i],":", classweight[i])

🔄 hand : 0.258777633289987
leg : 0.17509898812142544
hip : 1.1775147928994083
shoulder : 1.140401146131805
mixed : 1.0

# data augmentation

# normalization values for pretrained resnet on Imagenet
norm_mean = (0.485, 0.456, 0.406)
norm_std = (0.229, 0.224, 0.225)

batch_size = 128
validation_batch_size = 10
test_batch_size = 10

# We compute the weights of individual classes and convert them to tensors
class_weights = estimate_weights_mfb(label)
class_weights = torch.FloatTensor(class_weights)

transform_train = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(degrees=60),
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std),
])

transform_test = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])

class DenseNet121UNet(nn.Module):
    def __init__(self, n_classes=1):
        super(DenseNet121UNet, self).__init__()

        densenet = torchvision.models.densenet121(pretrained=True)

        # Use the 'features' submodule which includes conv, bn, relu, pool, and dense blocks
        self.encoder = densenet.features

        # You'll need to manually slice feature maps to simulate stages if you want skip connections
        # DenseNet has:
        # features.conv0 -> features.norm0 -> features.relu0 -> features.pool0
        # -> denseblock1 -> transition1 -> denseblock2 -> transition2 ...
        self.enc0 = nn.Sequential(
            densenet.features.conv0,
            densenet.features.norm0,
            densenet.features.relu0,
        )
        self.pool0 = densenet.features.pool0
        self.enc1 = densenet.features.denseblock1
        self.trans1 = densenet.features.transition1
        self.enc2 = densenet.features.denseblock2
        self.trans2 = densenet.features.transition2
        self.enc3 = densenet.features.denseblock3
        self.trans3 = densenet.features.transition3
        self.enc4 = densenet.features.denseblock4

        self.reduce_e3 = nn.Conv2d(1024, 512, kernel_size=1)

```

```

self.reduce_e2 = nn.Conv2d(512, 256, kernel_size=1)
self.reduce_e1 = nn.Conv2d(256, 128, kernel_size=1)
self.reduce_e0 = nn.Conv2d(64, 64, kernel_size=1) # Optional; already matches

```

```

# Decoder

```

```

self.up4 = self.up_block(1024, 512)
self.up3 = self.up_block(512, 256)
self.up2 = self.up_block(256, 128)
self.up1 = self.up_block(128, 64)

```

```

self.final_conv = nn.Conv2d(64, n_classes, kernel_size=1)

```

```

def up_block(self, in_channels, out_channels):
    return nn.Sequential(
        nn.ConvTranspose2d(in_channels, out_channels, kernel_size=2, stride=2),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
        nn.ReLU(inplace=True),
    )

```

```

def forward(self, x):
    e0 = self.enc0(x)
    p0 = self.pool0(e0)
    e1 = self.enc1(p0)
    t1 = self.trans1(e1)
    e2 = self.enc2(t1)
    t2 = self.trans2(e2)
    e3 = self.enc3(t2)
    t3 = self.trans3(e3)
    e4 = self.enc4(t3)

    d4 = self.up4(e4) + self.reduce_e3(e3)
    d3 = self.up3(d4) + self.reduce_e2(e2)
    d2 = self.up2(d3) + self.reduce_e1(e1)
    d1 = self.up1(d2) + self.reduce_e0(e0)

```

```

    return self.final_conv(d1)

```

```

model = DenseNet121UNet(n_classes=1).to(device)
criterion = nn.BCEWithLogitsLoss() # binary mask loss
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

```

```

➔ /usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
  warnings.warn(msg)

```

```

from torch.utils.data import Dataset
from pycocotools.coco import COCO
from PIL import Image
import torchvision.transforms.functional as TF
import numpy as np
import os

```

```

class FracAtlasCocoDataset(Dataset):
    def __init__(self, images_dir, ann_path, image_ids, transform=None):
        """
        images_dir: path to folder with X-ray images
        ann_path: path to COCO annotation JSON file
        image_ids: list of image file names to include (train/val split)
        transform: torchvision transforms to apply
        """
        self.images_dir = images_dir
        self.coco = COCO(ann_path)
        self.image_ids = [
            img['id'] for img in self.coco.dataset['images']
            if img['file_name'] in image_ids
        ]
        self.transform = transform

    def __len__(self):
        return len(self.image_ids)

```

```

def __getitem__(self, idx):
    img_id = self.image_ids[idx]
    img_info = self.coco.loadImgs(img_id)[0]
    img_path = os.path.join(self.images_dir, img_info['file_name'])

    # Load image
    image = Image.open(img_path).convert('RGB')

    # Create empty mask
    mask = np.zeros((img_info['height'], img_info['width']), dtype=np.uint8)

    # Get annotations for the image
    ann_ids = self.coco.getAnnIds(imgIds=img_id)
    anns = self.coco.loadAnns(ann_ids)

    for ann in anns:
        m = self.coco.annToMask(ann)
        mask = np.maximum(mask, m)

    # Convert to PIL Image for transforms
    mask = Image.fromarray(mask)

    if self.transform:
        # Apply the same transform to both image and mask
        image = self.transform(image)
        mask = TF.resize(mask, image.shape[1:]) # resize mask to match image (if needed)
        mask = TF.to_tensor(mask)
        mask = (mask > 0).float() # ensures binary (0 or 1)

    return image, mask

train_csv = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/train_combined.csv'
val_csv = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/valid_combined.csv'
test_csv = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/test_combined.csv'
ann_path = "/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Annotations/COCO_JSON/COCO_fracture_masks.json"
images_dir = "/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/images/sorted_images/Combined/all"

# Load train/val splits
train_ids = pd.read_csv(train_csv)['image_id'].tolist()
val_ids = pd.read_csv(val_csv)['image_id'].tolist()
test_ids = pd.read_csv(test_csv)['image_id'].tolist()

# Datasets
train_dataset = FracAtlasCocoDataset(images_dir, ann_path, train_ids, transform=transform_train)
val_dataset = FracAtlasCocoDataset(images_dir, ann_path, val_ids, transform=transform_test)
test_dataset = FracAtlasCocoDataset(images_dir, ann_path, test_ids, transform=transform_test)

# DataLoaders
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=4, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False)

↩ loading annotations into memory...
Done (t=0.01s)
creating index...
index created!
loading annotations into memory...
Done (t=0.01s)
creating index...
index created!
loading annotations into memory...
Done (t=0.01s)
creating index...
index created!

import torch
import numpy as np
from sklearn.metrics import average_precision_score

def compute_metrics(preds, targets, threshold=0.5, eps=1e-7):
    probs = torch.sigmoid(preds)
    preds_bin = (probs > threshold).float()

    preds_flat = preds_bin.view(-1)
    targets_flat = targets.view(-1)

    TP = (preds_flat * targets_flat).sum()

```

```

FP = (preds_flat * (1 - targets_flat)).sum()
FN = ((1 - preds_flat) * targets_flat).sum()
TN = ((1 - preds_flat) * (1 - targets_flat)).sum()

precision = TP / (TP + FP + eps)
recall = TP / (TP + FN + eps)
f1_score = 2 * precision * recall / (precision + recall + eps)
iou = TP / (TP + FP + FN + eps)
dice = 2 * TP / (2 * TP + FP + FN + eps)

ap = average_precision_score(
    targets_flat.cpu().numpy(),
    torch.sigmoid(preds).view(-1).cpu().numpy()
)

return {
    'IoU': iou.item(),
    'Precision': precision.item(),
    'Recall': recall.item(),
    'F1': f1_score.item(),
    'mAP': ap,
    'Dice': dice.item()
}

def test_model(model, test_loader, device):
    model.eval()
    all_metrics = []

    with torch.no_grad():
        for images, masks in test_loader:
            images = images.to(device)
            masks = masks.to(device)

            outputs = model(images)
            outputs = F.interpolate(outputs, size=masks.shape[2:], mode='bilinear', align_corners=False)

            metrics = compute_metrics(outputs, masks)
            all_metrics.append(metrics)

    avg_metrics = {
        key: np.mean([m[key] for m in all_metrics]) for key in all_metrics[0]
    }

    print("=== Test Metrics ===")
    for k, v in avg_metrics.items():
        print(f"{k}: {v:.4f}")
    return avg_metrics

```

```

# Estimate how imbalanced your dataset is
total_pixels = 0
fracture_pixels = 0

for _, mask in train_loader:
    fracture_pixels += mask.sum().item()
    total_pixels += mask.numel()
    break # remove break to compute over the full dataset if you want

```

```

fracture_ratio = fracture_pixels / total_pixels
print(f"Fracture pixel ratio: {fracture_ratio:.6f}")

```

➡ Fracture pixel ratio: 0.004135

```

# Compute pos_weight dynamically
pos_weight_value = 1.0 / (fracture_ratio + 1e-8)
print(f"Using pos_weight={pos_weight_value:.2f}")

```

```

# Set loss with pos_weight
pos_weight = torch.tensor([pos_weight_value]).to(device)
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

```

➡ Using pos_weight=241.81

```

def dice_score(pred, target, smooth=1.):
    pred = pred.view(-1)
    target = target.view(-1)
    intersection = (pred * target).sum()
    return (2. * intersection + smooth) / (pred.sum() + target.sum() + smooth)

def iou_score(pred, target, smooth=1.):
    pred = pred.view(-1)
    target = target.view(-1)
    intersection = (pred * target).sum()
    union = pred.sum() + target.sum() - intersection
    return (intersection + smooth) / (union + smooth)

def precision_score(pred, target, eps=1e-8):
    tp = (pred * target).sum()
    fp = (pred * (1 - target)).sum()
    return (tp + eps) / (tp + fp + eps)

def recall_score(pred, target, eps=1e-8):
    tp = (pred * target).sum()
    fn = ((1 - pred) * target).sum()
    return (tp + eps) / (tp + fn + eps)

def f1_score(pred, target, eps=1e-8):
    precision = precision_score(pred, target, eps)
    recall = recall_score(pred, target, eps)
    return (2 * precision * recall + eps) / (precision + recall + eps)

def accuracy_score(pred, target, eps=1e-8):
    correct = (pred == target).float().sum()
    total = target.numel()
    return correct / (total + eps)

def save_checkpoint(model, optimizer, epoch, path="checkpoint.pth"):
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
    }, path)

def compute_confusion_matrix(preds, targets):
    """
    Computes TP, TN, FP, FN for binary segmentation.
    Args:
        preds (Tensor): shape (B, 1, H, W)
        targets (Tensor): shape (B, 1, H, W)
    Returns:
        Dictionary with TP, TN, FP, FN
    """
    preds = preds.view(-1)
    targets = targets.view(-1)

    TP = ((preds == 1) & (targets == 1)).sum().item()
    TN = ((preds == 0) & (targets == 0)).sum().item()
    FP = ((preds == 1) & (targets == 0)).sum().item()
    FN = ((preds == 0) & (targets == 1)).sum().item()

    return {"TP": TP, "TN": TN, "FP": FP, "FN": FN}

def compute_mAP(preds, targets, thresholds=np.arange(0.5, 1.0, 0.05), eps=1e-8):
    """
    Computes mean Average Precision (mAP) across thresholds for binary segmentation.
    preds and targets should be torch tensors of shape (B, 1, H, W)
    """
    preds = preds.view(-1).cpu().numpy()
    targets = targets.view(-1).cpu().numpy()

    APs = []

    for thresh in thresholds:
        binarized = (preds >= thresh).astype(np.uint8)

        TP = np.logical_and(binarized == 1, targets == 1).sum()
        FP = np.logical_and(binarized == 1, targets == 0).sum()
        FN = np.logical_and(binarized == 0, targets == 1).sum()

        precision = TP / (TP + FP + eps)

```

```

    recall = TP / (TP + FN + eps)

    AP = precision * recall # proxy for real AP (simplified)
    APs.append(AP)

mAP = np.mean(APs)
return mAP

import torch.nn.functional as F

train_losses = []
train_dices = []
train_iou = []
train_f1s = []
train_precisions = []
train_recalls = []
train_accuracies = []

val_losses = []
val_dices = []
val_iou = []
val_f1s = []
val_precisions = []
val_recalls = []
val_accuracies = []
val_maps = []

num_epochs = 50
best_val_dice = 0.0

for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0
    total_dice = 0.0
    total_iou = 0.0
    total_f1 = 0.0
    total_precision = 0.0
    total_recall = 0.0
    total_acc = 0.0

    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        outputs = F.interpolate(outputs, size=masks.shape[2:], mode='bilinear', align_corners=False)
        loss = criterion(outputs, masks)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

        with torch.no_grad():
            probs = torch.sigmoid(outputs)
            preds = (probs > 0.5).float()
            total_dice += dice_score(preds, masks).item()
            total_iou += iou_score(preds, masks).item()
            total_f1 += f1_score(preds, masks).item()
            total_precision += precision_score(preds, masks).item()
            total_recall += recall_score(preds, masks).item()
            total_acc += accuracy_score(preds, masks).item()

    conf_matrix = {"TP": 0, "TN": 0, "FP": 0, "FN": 0}

    # Average training metrics for this epoch
    avg_train_loss = total_loss / len(train_loader)
    avg_train_dice = total_dice / len(train_loader)
    avg_train_iou = total_iou / len(train_loader)
    avg_train_f1 = total_f1 / len(train_loader)
    avg_train_precision = total_precision / len(train_loader)
    avg_train_recall = total_recall / len(train_loader)
    avg_train_acc = total_acc / len(train_loader)

    # Store training metrics
    train_losses.append(avg_train_loss)
    train_dices.append(avg_train_dice)
    train_iou.append(avg_train_iou)

```

```

train_f1s.append(avg_train_f1)
train_precisions.append(avg_train_precision)
train_recalls.append(avg_train_recall)
train_accuracies.append(avg_train_acc)

# --- VALIDATION ---
model.eval()
total_dice = 0.0
total_iou = 0.0
total_f1 = 0.0
total_precision = 0.0
total_recall = 0.0
total_acc = 0.0
total_loss_val = 0.0

with torch.no_grad():
    all_preds = []
    all_targets = []
    for images, masks in val_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        outputs = F.interpolate(outputs, size=masks.shape[2:], mode='bilinear', align_corners=False)
        loss_val = criterion(outputs, masks)
        total_loss_val += loss_val.item()
        probs = torch.sigmoid(outputs)
        preds = (probs > 0.5).float()

        batch_matrix = compute_confusion_matrix(preds, masks)
        for k in conf_matrix:
            conf_matrix[k] += batch_matrix[k]

        total_dice += dice_score(preds, masks).item()
        total_iou += iou_score(preds, masks).item()
        total_f1 += f1_score(preds, masks).item()
        total_precision += precision_score(preds, masks).item()
        total_recall += recall_score(preds, masks).item()
        total_acc += accuracy_score(preds, masks).item()
        all_preds.append(probs.cpu())
        all_targets.append(masks.cpu())

avg_val_loss = total_loss_val / len(val_loader)
avg_val_dice = total_dice / len(val_loader)
avg_val_iou = total_iou / len(val_loader)
avg_val_f1 = total_f1 / len(val_loader)
avg_val_precision = total_precision / len(val_loader)
avg_val_recall = total_recall / len(val_loader)
avg_val_acc = total_acc / len(val_loader)

val_losses.append(avg_val_loss)
val_dices.append(avg_val_dice)
val_iou.append(avg_val_iou)
val_f1s.append(avg_val_f1)
val_precisions.append(avg_val_precision)
val_recalls.append(avg_val_recall)
val_accuracies.append(avg_val_acc)

all_preds = torch.cat(all_preds, dim=0)
all_targets = torch.cat(all_targets, dim=0)

val_map = compute_mAP(all_preds, all_targets)
val_maps.append(val_map)

print(f"Epoch {epoch+1}/{num_epochs} | "
      f"Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f} | "
      f"Dice: {avg_val_dice:.4f} | IoU: {avg_val_iou:.4f} | "
      f"mAP: {val_map:.4f} | "
      f"F1: {avg_val_f1:.4f} | Precision: {avg_val_precision:.4f} | Recall: {avg_val_recall:.4f} | Acc: {avg_val_acc:.4f}")

if avg_val_dice > best_val_dice:
    best_val_dice = avg_val_dice
    save_checkpoint(model, optimizer, epoch)
    print("Checkpoint saved!")

```



Epoch 7/50 | Train Loss: 0.7202 | Val Loss: 0.5404 | Dice: 0.8414 | IoU: 0.8212 | mAP: 0.8410 | F1: 0.8414 | Precision: 0.82


```

Epoch 7/50 | Train Loss: 0.7203 | Val Loss: 0.5494 | Dice: 0.0414 | IoU: 0.0212 | mAP: 0.0410 | F1: 0.0414 | Precision: 0.02
Epoch 8/50 | Train Loss: 0.7018 | Val Loss: 0.5845 | Dice: 0.0380 | IoU: 0.0195 | mAP: 0.0354 | F1: 0.0380 | Precision: 0.01
Epoch 9/50 | Train Loss: 0.6776 | Val Loss: 0.5171 | Dice: 0.0465 | IoU: 0.0240 | mAP: 0.0373 | F1: 0.0465 | Precision: 0.02
Checkpoint saved!
Epoch 10/50 | Train Loss: 0.6449 | Val Loss: 0.4885 | Dice: 0.0472 | IoU: 0.0243 | mAP: 0.0425 | F1: 0.0471 | Precision: 0.0
Checkpoint saved!
Epoch 11/50 | Train Loss: 0.5920 | Val Loss: 0.5404 | Dice: 0.0477 | IoU: 0.0245 | mAP: 0.0418 | F1: 0.0476 | Precision: 0.0
Checkpoint saved!
Epoch 12/50 | Train Loss: 0.6316 | Val Loss: 0.5056 | Dice: 0.0521 | IoU: 0.0269 | mAP: 0.0436 | F1: 0.0521 | Precision: 0.0
Checkpoint saved!
Epoch 13/50 | Train Loss: 0.5845 | Val Loss: 0.5245 | Dice: 0.0448 | IoU: 0.0230 | mAP: 0.0370 | F1: 0.0448 | Precision: 0.0
Epoch 14/50 | Train Loss: 0.6185 | Val Loss: 0.4880 | Dice: 0.0410 | IoU: 0.0211 | mAP: 0.0431 | F1: 0.0410 | Precision: 0.0
Epoch 15/50 | Train Loss: 0.5859 | Val Loss: 0.4709 | Dice: 0.0517 | IoU: 0.0267 | mAP: 0.0384 | F1: 0.0516 | Precision: 0.0
Epoch 16/50 | Train Loss: 0.5627 | Val Loss: 0.4603 | Dice: 0.0513 | IoU: 0.0265 | mAP: 0.0430 | F1: 0.0513 | Precision: 0.0
Epoch 17/50 | Train Loss: 0.5511 | Val Loss: 0.4676 | Dice: 0.0659 | IoU: 0.0343 | mAP: 0.0561 | F1: 0.0659 | Precision: 0.0
Checkpoint saved!
Epoch 18/50 | Train Loss: 0.5382 | Val Loss: 0.4741 | Dice: 0.0597 | IoU: 0.0310 | mAP: 0.0460 | F1: 0.0596 | Precision: 0.0
Epoch 19/50 | Train Loss: 0.5489 | Val Loss: 0.4664 | Dice: 0.0550 | IoU: 0.0284 | mAP: 0.0493 | F1: 0.0550 | Precision: 0.0
Epoch 20/50 | Train Loss: 0.5284 | Val Loss: 0.4408 | Dice: 0.0602 | IoU: 0.0313 | mAP: 0.0495 | F1: 0.0601 | Precision: 0.0
Epoch 21/50 | Train Loss: 0.5550 | Val Loss: 0.5673 | Dice: 0.0602 | IoU: 0.0313 | mAP: 0.0387 | F1: 0.0601 | Precision: 0.0
Epoch 22/50 | Train Loss: 0.4902 | Val Loss: 0.4349 | Dice: 0.0654 | IoU: 0.0341 | mAP: 0.0517 | F1: 0.0653 | Precision: 0.0
Epoch 23/50 | Train Loss: 0.5299 | Val Loss: 0.4511 | Dice: 0.0717 | IoU: 0.0375 | mAP: 0.0477 | F1: 0.0716 | Precision: 0.0
Checkpoint saved!
Epoch 24/50 | Train Loss: 0.4700 | Val Loss: 0.4329 | Dice: 0.0639 | IoU: 0.0333 | mAP: 0.0521 | F1: 0.0639 | Precision: 0.0
Epoch 25/50 | Train Loss: 0.4651 | Val Loss: 0.4969 | Dice: 0.0714 | IoU: 0.0374 | mAP: 0.0501 | F1: 0.0713 | Precision: 0.0
Epoch 26/50 | Train Loss: 0.4586 | Val Loss: 0.4103 | Dice: 0.0772 | IoU: 0.0406 | mAP: 0.0605 | F1: 0.0771 | Precision: 0.0
Checkpoint saved!
Epoch 27/50 | Train Loss: 0.4841 | Val Loss: 0.4525 | Dice: 0.0657 | IoU: 0.0342 | mAP: 0.0537 | F1: 0.0656 | Precision: 0.0
Epoch 28/50 | Train Loss: 0.4584 | Val Loss: 0.4703 | Dice: 0.0719 | IoU: 0.0376 | mAP: 0.0589 | F1: 0.0719 | Precision: 0.0
Epoch 29/50 | Train Loss: 0.4549 | Val Loss: 0.4749 | Dice: 0.0648 | IoU: 0.0338 | mAP: 0.0496 | F1: 0.0648 | Precision: 0.0
Epoch 30/50 | Train Loss: 0.4648 | Val Loss: 0.4898 | Dice: 0.0619 | IoU: 0.0322 | mAP: 0.0543 | F1: 0.0619 | Precision: 0.0
Epoch 31/50 | Train Loss: 0.4473 | Val Loss: 0.5288 | Dice: 0.0599 | IoU: 0.0311 | mAP: 0.0437 | F1: 0.0598 | Precision: 0.0
Epoch 32/50 | Train Loss: 0.4378 | Val Loss: 0.4897 | Dice: 0.0638 | IoU: 0.0335 | mAP: 0.0508 | F1: 0.0638 | Precision: 0.0
Epoch 33/50 | Train Loss: 0.4424 | Val Loss: 0.4572 | Dice: 0.0715 | IoU: 0.0374 | mAP: 0.0525 | F1: 0.0715 | Precision: 0.0
Epoch 34/50 | Train Loss: 0.3974 | Val Loss: 0.4758 | Dice: 0.0795 | IoU: 0.0418 | mAP: 0.0593 | F1: 0.0795 | Precision: 0.0
Checkpoint saved!
Epoch 35/50 | Train Loss: 0.3962 | Val Loss: 0.6978 | Dice: 0.0781 | IoU: 0.0410 | mAP: 0.0587 | F1: 0.0780 | Precision: 0.0
Epoch 36/50 | Train Loss: 0.4129 | Val Loss: 0.4612 | Dice: 0.0822 | IoU: 0.0433 | mAP: 0.0653 | F1: 0.0822 | Precision: 0.0
Checkpoint saved!
Epoch 37/50 | Train Loss: 0.3987 | Val Loss: 0.4372 | Dice: 0.0541 | IoU: 0.0280 | mAP: 0.0444 | F1: 0.0541 | Precision: 0.0
Epoch 38/50 | Train Loss: 0.4080 | Val Loss: 0.4275 | Dice: 0.0803 | IoU: 0.0422 | mAP: 0.0615 | F1: 0.0802 | Precision: 0.0
Epoch 39/50 | Train Loss: 0.3764 | Val Loss: 0.5029 | Dice: 0.0979 | IoU: 0.0519 | mAP: 0.0684 | F1: 0.0979 | Precision: 0.0
Checkpoint saved!
Epoch 40/50 | Train Loss: 0.3809 | Val Loss: 0.4768 | Dice: 0.0616 | IoU: 0.0320 | mAP: 0.0494 | F1: 0.0615 | Precision: 0.0
Epoch 41/50 | Train Loss: 0.3950 | Val Loss: 0.4256 | Dice: 0.0725 | IoU: 0.0380 | mAP: 0.0563 | F1: 0.0725 | Precision: 0.0
Epoch 42/50 | Train Loss: 0.4494 | Val Loss: 0.5541 | Dice: 0.0573 | IoU: 0.0297 | mAP: 0.0458 | F1: 0.0573 | Precision: 0.0
Epoch 43/50 | Train Loss: 0.4600 | Val Loss: 0.5739 | Dice: 0.0775 | IoU: 0.0407 | mAP: 0.0573 | F1: 0.0775 | Precision: 0.0
Epoch 44/50 | Train Loss: 0.4021 | Val Loss: 0.5174 | Dice: 0.0832 | IoU: 0.0439 | mAP: 0.0541 | F1: 0.0831 | Precision: 0.0
Epoch 45/50 | Train Loss: 0.4321 | Val Loss: 0.5423 | Dice: 0.0613 | IoU: 0.0319 | mAP: 0.0433 | F1: 0.0613 | Precision: 0.0
Epoch 46/50 | Train Loss: 0.4091 | Val Loss: 0.4149 | Dice: 0.0735 | IoU: 0.0385 | mAP: 0.0634 | F1: 0.0735 | Precision: 0.0
Epoch 47/50 | Train Loss: 0.3903 | Val Loss: 0.4895 | Dice: 0.0742 | IoU: 0.0388 | mAP: 0.0575 | F1: 0.0741 | Precision: 0.0
Epoch 48/50 | Train Loss: 0.3625 | Val Loss: 0.3924 | Dice: 0.0839 | IoU: 0.0442 | mAP: 0.0646 | F1: 0.0838 | Precision: 0.0
Epoch 49/50 | Train Loss: 0.3512 | Val Loss: 0.5403 | Dice: 0.0765 | IoU: 0.0402 | mAP: 0.0560 | F1: 0.0764 | Precision: 0.0
Epoch 50/50 | Train Loss: 0.3595 | Val Loss: 0.4523 | Dice: 0.0735 | IoU: 0.0385 | mAP: 0.0550 | F1: 0.0734 | Precision: 0.0

```

```

import matplotlib.pyplot as plt

epochs = list(range(1, num_epochs + 1))

plt.figure(figsize=(18, 12))

# --- Loss Curve ---
plt.subplot(2, 3, 1)
plt.plot(epochs, train_losses, 'o-', color='orange', label='Train')
plt.plot(epochs, val_losses, 'o-', color='blue', label='Validation')
plt.title("Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.legend()

# --- Dice Curve ---
plt.subplot(2, 3, 2)
plt.plot(epochs, train_dices, 'o-', color='orange', label='Train')
plt.plot(epochs, val_dices, 'o-', color='blue', label='Validation')
plt.title("Dice Score")
plt.xlabel("Epoch")
plt.ylabel("Dice")
plt.grid(True)
plt.legend()

# --- IoU Curve ---

```

```

plt.subplot(2, 3, 3)
plt.plot(epochs, train_iou, 'o-', color='orange', label='Train')
plt.plot(epochs, val_iou, 'o-', color='blue', label='Validation')
plt.title("IoU Score")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

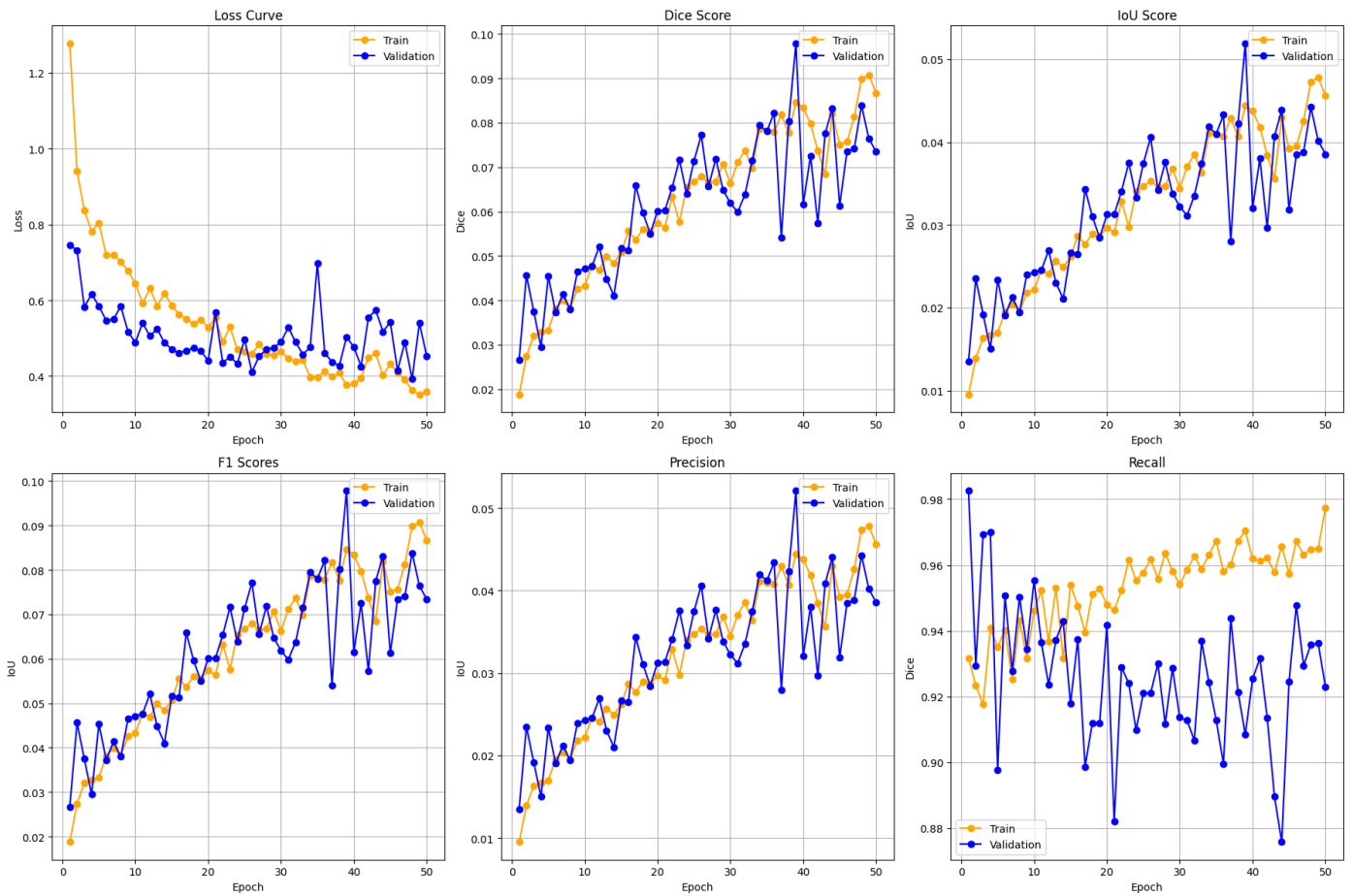
# --- F1 Curve ---
plt.subplot(2, 3, 4)
plt.plot(epochs, train_f1s, 'o-', color='orange', label='Train')
plt.plot(epochs, val_f1s, 'o-', color='blue', label='Validation')
plt.title("F1 Scores")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

# --- Precision Curve ---
plt.subplot(2, 3, 5)
plt.plot(epochs, train_precisions, 'o-', color='orange', label='Train')
plt.plot(epochs, val_precisions, 'o-', color='blue', label='Validation')
plt.title("Precision")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

# --- Recall Curve ---
plt.subplot(2, 3, 6)
plt.plot(epochs, train_recalls, 'o-', color='orange', label='Train')
plt.plot(epochs, val_recalls, 'o-', color='blue', label='Validation')
plt.title("Recall")
plt.xlabel("Epoch")
plt.ylabel("Dice")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import torchvision.transforms.functional as TF

def show_prediction_with_box(model, dataset, index=0, threshold=0.5):
    model.eval()
    with torch.no_grad():
        # Get image and corresponding image_id
        image, _ = dataset[index]
        image_id = dataset.image_ids[index]
        image_info = dataset.coco.loadImgs(image_id)[0]
        filename = image_info['file_name']

    print(f"🖼️ Image ID: {filename}")

    input_tensor = image.unsqueeze(0).to(device)

    # Run model
    output = model(input_tensor)
    output = F.interpolate(output, size=image.shape[1:], mode='bilinear', align_corners=False)
    probs = torch.sigmoid(output)
    pred_mask = (probs > threshold).float().cpu().numpy()[0, 0]

    # Convert to bounding boxes using OpenCV
    mask_uint8 = (pred_mask * 255).astype(np.uint8)
    contours, _ = cv2.findContours(mask_uint8, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    # Original image as NumPy for plotting
```

```
img_np = image.permute(1, 2, 0).cpu().numpy()
img_np = (img_np - img_np.min()) / (img_np.max() - img_np.min()) # Normalize for display
img_disp = (img_np * 255).astype(np.uint8).copy()
```

```
# Draw bounding boxes
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img_disp, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

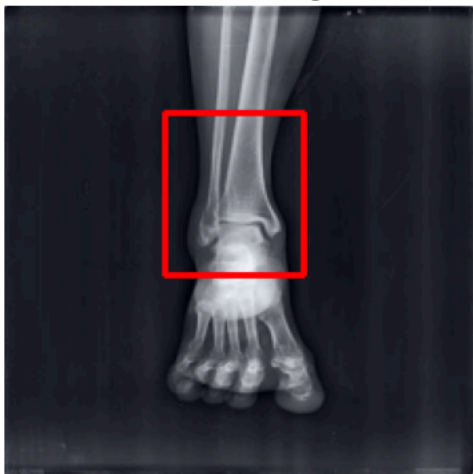
```
# Show image and predicted mask
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(img_disp)
ax[0].set_title("Predicted Bounding Box")
ax[0].axis("off")
```

```
ax[1].imshow(pred_mask, cmap='gray')
ax[1].set_title("Predicted Mask")
ax[1].axis("off")
plt.show()
```

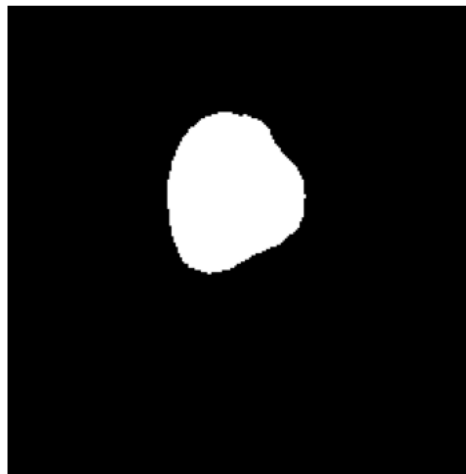
```
# Show bounding box prediction on test image
show_prediction_with_box(model, val_dataset, index=11)
show_prediction_with_box(model, val_dataset, index=10)
```

🔄 🖼️ Image ID: IMG0003803.jpg

Predicted Bounding Box



Predicted Mask



🖼️ Image ID: IMG0003798.jpg

Predicted Bounding Box



Predicted Mask



```
import matplotlib.pyplot as plt
import cv2
from PIL import Image
def show_coco_box_from_original_image(dataset, index=0, image_root="/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas

    image_id = dataset.image_ids[index]
    image_info = dataset.coco.loadImgs(image_id)[0]
```

```

filename = image_info['file_name']
filepath = os.path.join(image_root, filename)

print(f"🖼️ Raw image with COCO boxes: {filename}")

# Load original image
img = cv2.imread(filepath)
if img is None:
    print("❌ Failed to load image.")
    return
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Get annotations and draw boxes
ann_ids = dataset.coco.getAnnIds(imgIds=image_id)
anns = dataset.coco.loadAnns(ann_ids)

for i, ann in enumerate(anns):
    x, y, w, h = map(int, ann['bbox']) # COCO box format
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2) # 🟢 Green GT box
    print(f"🟢 GT Box {i+1}: x={x}, y={y}, w={w}, h={h}")

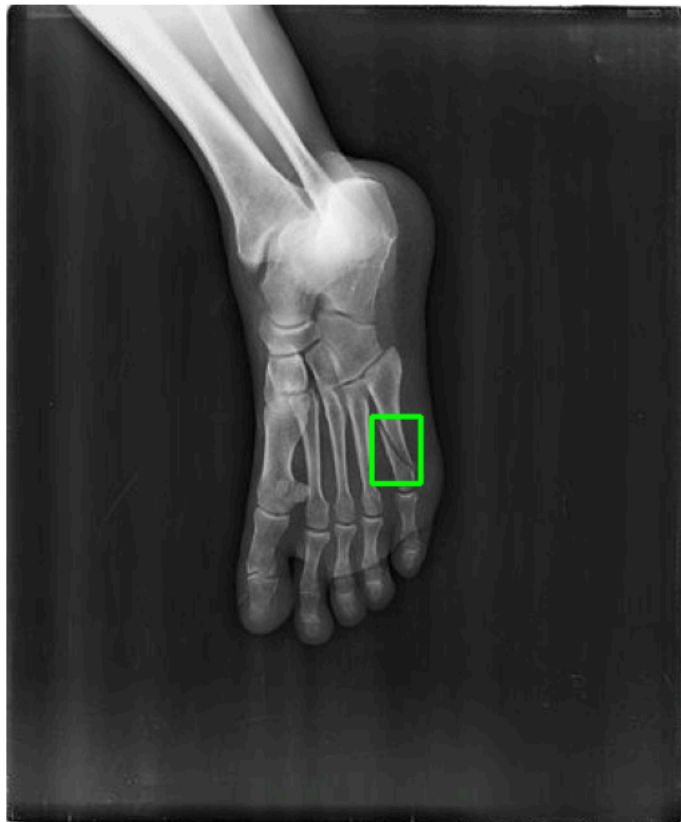
plt.figure(figsize=(8, 8))
plt.imshow(img)
plt.title("🟢 COCO GT Boxes on Original Image")
plt.axis("off")
plt.show()

show_coco_box_from_original_image(val_dataset, index=10)
show_coco_box_from_original_image(val_dataset, index=11)

```

Raw image with COCO boxes: IMG0003798.jpg
GT Box 1: x=201, y=227, w=27, h=37
/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128309 (\N{LARGE BLUE CIRCLE}) missing from font. fig.canvas.print_figure(bytes_io, **kw)

□ COCO GT Boxes on Original Image



Raw image with COCO boxes: IMG0003803.jpg
GT Box 1: x=154, y=191, w=16, h=23

□ COCO GT Boxes on Original Image



```
test_metrics = test_model(model, test_loader, device)
```

```
➦ === Test Metrics ===  
IoU: 0.0255  
Precision: 0.0257  
Recall: 0.8852  
F1: 0.0494  
mAP: 0.1532  
Dice: 0.0494
```

```
# Create a new train .csv file and apply oversampling
```

```
# Load the CSVs
```

```
fracture_df = pd.read_csv('/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/train.csv')  
nonfracture_df = pd.read_csv('/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/nf_train.csv')
```

```
# Oversample fracture images (4x duplication)
```

```
oversampled_fractures = pd.concat([fracture_df] * 4, ignore_index=True)
```

```
# Combine and shuffle
```

```
combined_df = pd.concat([oversampled_fractures, nonfracture_df], ignore_index=True)  
combined_df = combined_df.sample(frac=1, random_state=42) # shuffle rows
```

```
# Save the result
```

```
combined_df[['image_id']].to_csv('/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/train_cc
```

```
print(f"Combined dataset saved with {len(combined_df)} entries.")
```

```
➦ Combined dataset saved with 2296 entries.
```

```
train_csv = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/train_combined.csv'  
val_csv = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/valid_combined.csv'  
test_csv = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/test_combined.csv'  
ann_path = "/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Annotations/COCO JSON/COCO_fracture_masks.json"  
images_dir = "/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/images/sorted_images/Combined/all"
```

```
# Load train/val splits
```

```
train_ids = pd.read_csv(train_csv)['image_id'].tolist()  
val_ids = pd.read_csv(val_csv)['image_id'].tolist()  
test_ids = pd.read_csv(test_csv)['image_id'].tolist()
```

```
# Datasets
```

```
train_dataset = FracAtlasCocoDataset(images_dir, ann_path, train_ids, transform=transform_train)  
val_dataset = FracAtlasCocoDataset(images_dir, ann_path, val_ids, transform=transform_test)  
test_dataset = FracAtlasCocoDataset(images_dir, ann_path, test_ids, transform=transform_test)
```

```
# DataLoaders
```

```
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)  
val_loader = DataLoader(val_dataset, batch_size=4, shuffle=False)  
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False)
```

```
➦ loading annotations into memory...  
Done (t=0.01s)  
creating index...  
index created!  
loading annotations into memory...  
Done (t=0.03s)  
creating index...  
index created!  
loading annotations into memory...  
Done (t=0.01s)  
creating index...  
index created!
```

```

# Estimate how imbalanced your dataset is
total_pixels = 0
fracture_pixels = 0

for _, mask in train_loader:
    fracture_pixels += mask.sum().item()
    total_pixels += mask.numel()
    break # remove break to compute over the full dataset if you want

fracture_ratio = fracture_pixels / total_pixels
print(f"Fracture pixel ratio: {fracture_ratio:.6f}")

➡ Fracture pixel ratio: 0.002795

# Compute pos_weight dynamically
pos_weight_value = 1.0 / (fracture_ratio + 1e-8)
print(f"Using pos_weight={pos_weight_value:.2f}")

# Set loss with pos_weight
pos_weight = torch.tensor([pos_weight_value]).to(device)
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

➡ Using pos_weight=357.76

import torch.nn.functional as F

train_losses = []
train_dices = []
train_iou = []
train_f1s = []
train_precisions = []
train_recalls = []
train_accuracies = []

val_losses = []
val_dices = []
val_iou = []
val_f1s = []
val_precisions = []
val_recalls = []
val_accuracies = []
val_maps = []

num_epochs = 50
best_val_dice = 0.0

for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0
    total_dice = 0.0
    total_iou = 0.0
    total_f1 = 0.0
    total_precision = 0.0
    total_recall = 0.0
    total_acc = 0.0

    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        outputs = F.interpolate(outputs, size=masks.shape[2:], mode='bilinear', align_corners=False)
        loss = criterion(outputs, masks)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    with torch.no_grad():
        probs = torch.sigmoid(outputs)
        preds = (probs > 0.5).float()
        total_dice += dice_score(preds, masks).item()
        total_iou += iou_score(preds, masks).item()
        total_f1 += f1_score(preds, masks).item()
        total_precision += precision_score(preds, masks).item()
        total_recall += recall_score(preds, masks).item()
        total_acc += accuracy_score(preds, masks).item()

```



```

conf_matrix = {"TP": 0, "TN": 0, "FP": 0, "FN": 0}

# Average training metrics for this epoch
avg_train_loss = total_loss / len(train_loader)
avg_train_dice = total_dice / len(train_loader)
avg_train_iou = total_iou / len(train_loader)
avg_train_f1 = total_f1 / len(train_loader)
avg_train_precision = total_precision / len(train_loader)
avg_train_recall = total_recall / len(train_loader)
avg_train_acc = total_acc / len(train_loader)

# Store training metrics
train_losses.append(avg_train_loss)
train_dices.append(avg_train_dice)
train_iou.append(avg_train_iou)
train_f1s.append(avg_train_f1)
train_precisions.append(avg_train_precision)
train_recalls.append(avg_train_recall)
train_accuracies.append(avg_train_acc)

# --- VALIDATION ---
model.eval()
total_dice = 0.0
total_iou = 0.0
total_f1 = 0.0
total_precision = 0.0
total_recall = 0.0
total_acc = 0.0
total_loss_val = 0.0

with torch.no_grad():
    all_preds = []
    all_targets = []
    for images, masks in val_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        outputs = F.interpolate(outputs, size=masks.shape[2:], mode='bilinear', align_corners=False)
        loss_val = criterion(outputs, masks)
        total_loss_val += loss_val.item()
        probs = torch.sigmoid(outputs)
        preds = (probs > 0.5).float()

        batch_matrix = compute_confusion_matrix(preds, masks)
        for k in conf_matrix:
            conf_matrix[k] += batch_matrix[k]

        total_dice += dice_score(preds, masks).item()
        total_iou += iou_score(preds, masks).item()
        total_f1 += f1_score(preds, masks).item()
        total_precision += precision_score(preds, masks).item()
        total_recall += recall_score(preds, masks).item()
        total_acc += accuracy_score(preds, masks).item()
        all_preds.append(probs.cpu())
        all_targets.append(masks.cpu())

avg_val_loss = total_loss_val / len(val_loader)
avg_val_dice = total_dice / len(val_loader)
avg_val_iou = total_iou / len(val_loader)
avg_val_f1 = total_f1 / len(val_loader)
avg_val_precision = total_precision / len(val_loader)
avg_val_recall = total_recall / len(val_loader)
avg_val_acc = total_acc / len(val_loader)

val_losses.append(avg_val_loss)
val_dices.append(avg_val_dice)
val_iou.append(avg_val_iou)
val_f1s.append(avg_val_f1)
val_precisions.append(avg_val_precision)
val_recalls.append(avg_val_recall)
val_accuracies.append(avg_val_acc)

all_preds = torch.cat(all_preds, dim=0)
all_targets = torch.cat(all_targets, dim=0)

val_map = compute_mAP(all_preds, all_targets)
val_maps.append(val_map)

```

```

print(f"Epoch {epoch+1}/{num_epochs} | "
      f"Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f} | "
      f"Dice: {avg_val_dice:.4f} | IoU: {avg_val_iou:.4f} | "
      f"mAP: {val_map:.4f} | "
      f"F1: {avg_val_f1:.4f} | Precision: {avg_val_precision:.4f} | Recall: {avg_val_recall:.4f} | Acc: {avg_val_acc:.4f}")

if avg_val_dice > best_val_dice:
    best_val_dice = avg_val_dice
    save_checkpoint(model, optimizer, epoch)
    print("Checkpoint saved!")

```

```

Epoch 1/50 | Train Loss: 0.3933 | Val Loss: 0.6257 | Dice: 0.0898 | IoU: 0.0475 | mAP: 0.0634 | F1: 0.0898 | Precision: 0.04
Checkpoint saved!
Epoch 2/50 | Train Loss: 0.4662 | Val Loss: 0.5465 | Dice: 0.0714 | IoU: 0.0373 | mAP: 0.0468 | F1: 0.0713 | Precision: 0.03
Epoch 3/50 | Train Loss: 0.4431 | Val Loss: 0.5459 | Dice: 0.0806 | IoU: 0.0424 | mAP: 0.0544 | F1: 0.0805 | Precision: 0.04
Epoch 4/50 | Train Loss: 0.3928 | Val Loss: 0.5581 | Dice: 0.0838 | IoU: 0.0443 | mAP: 0.0584 | F1: 0.0837 | Precision: 0.04
Epoch 5/50 | Train Loss: 0.3776 | Val Loss: 0.6789 | Dice: 0.0711 | IoU: 0.0373 | mAP: 0.0462 | F1: 0.0711 | Precision: 0.03
Epoch 6/50 | Train Loss: 0.3732 | Val Loss: 0.7949 | Dice: 0.0803 | IoU: 0.0423 | mAP: 0.0495 | F1: 0.0802 | Precision: 0.04
Epoch 7/50 | Train Loss: 0.3917 | Val Loss: 0.8507 | Dice: 0.0824 | IoU: 0.0434 | mAP: 0.0513 | F1: 0.0823 | Precision: 0.04
Epoch 8/50 | Train Loss: 0.3912 | Val Loss: 0.7407 | Dice: 0.0729 | IoU: 0.0381 | mAP: 0.0500 | F1: 0.0728 | Precision: 0.03
Epoch 9/50 | Train Loss: 0.3877 | Val Loss: 0.6736 | Dice: 0.0963 | IoU: 0.0513 | mAP: 0.0677 | F1: 0.0963 | Precision: 0.05
Checkpoint saved!
Epoch 10/50 | Train Loss: 0.4105 | Val Loss: 0.6824 | Dice: 0.0778 | IoU: 0.0410 | mAP: 0.0529 | F1: 0.0778 | Precision: 0.0
Epoch 11/50 | Train Loss: 0.3757 | Val Loss: 0.7251 | Dice: 0.0792 | IoU: 0.0417 | mAP: 0.0477 | F1: 0.0792 | Precision: 0.0
Epoch 12/50 | Train Loss: 0.3561 | Val Loss: 0.7538 | Dice: 0.0861 | IoU: 0.0455 | mAP: 0.0543 | F1: 0.0861 | Precision: 0.0
Epoch 13/50 | Train Loss: 0.3760 | Val Loss: 0.7488 | Dice: 0.0923 | IoU: 0.0490 | mAP: 0.0604 | F1: 0.0923 | Precision: 0.0
Epoch 14/50 | Train Loss: 0.3690 | Val Loss: 0.5768 | Dice: 0.0942 | IoU: 0.0500 | mAP: 0.0671 | F1: 0.0941 | Precision: 0.0
Epoch 15/50 | Train Loss: 0.3737 | Val Loss: 0.5176 | Dice: 0.0963 | IoU: 0.0512 | mAP: 0.0704 | F1: 0.0962 | Precision: 0.0
Epoch 16/50 | Train Loss: 0.3769 | Val Loss: 0.5537 | Dice: 0.0775 | IoU: 0.0407 | mAP: 0.0551 | F1: 0.0774 | Precision: 0.0
Epoch 17/50 | Train Loss: 0.3571 | Val Loss: 0.8188 | Dice: 0.0903 | IoU: 0.0480 | mAP: 0.0665 | F1: 0.0903 | Precision: 0.0
Epoch 18/50 | Train Loss: 0.3441 | Val Loss: 0.7744 | Dice: 0.0946 | IoU: 0.0504 | mAP: 0.0624 | F1: 0.0946 | Precision: 0.0
Epoch 19/50 | Train Loss: 0.3648 | Val Loss: 0.5566 | Dice: 0.0894 | IoU: 0.0474 | mAP: 0.0615 | F1: 0.0893 | Precision: 0.0
Epoch 20/50 | Train Loss: 0.3754 | Val Loss: 0.7687 | Dice: 0.0961 | IoU: 0.0512 | mAP: 0.0622 | F1: 0.0960 | Precision: 0.0
Epoch 21/50 | Train Loss: 0.3322 | Val Loss: 0.6243 | Dice: 0.0945 | IoU: 0.0502 | mAP: 0.0652 | F1: 0.0944 | Precision: 0.0
Epoch 22/50 | Train Loss: 0.3961 | Val Loss: 0.5083 | Dice: 0.0889 | IoU: 0.0471 | mAP: 0.0630 | F1: 0.0889 | Precision: 0.0
Epoch 23/50 | Train Loss: 0.3973 | Val Loss: 0.6599 | Dice: 0.0848 | IoU: 0.0447 | mAP: 0.0576 | F1: 0.0847 | Precision: 0.0
Epoch 24/50 | Train Loss: 0.3631 | Val Loss: 0.5606 | Dice: 0.1126 | IoU: 0.0605 | mAP: 0.0770 | F1: 0.1125 | Precision: 0.0
Checkpoint saved!
Epoch 25/50 | Train Loss: 0.3400 | Val Loss: 0.5443 | Dice: 0.0994 | IoU: 0.0529 | mAP: 0.0675 | F1: 0.0993 | Precision: 0.0
Epoch 26/50 | Train Loss: 0.3195 | Val Loss: 0.7753 | Dice: 0.1034 | IoU: 0.0553 | mAP: 0.0662 | F1: 0.1034 | Precision: 0.0
Epoch 27/50 | Train Loss: 0.3123 | Val Loss: 1.0627 | Dice: 0.0989 | IoU: 0.0527 | mAP: 0.0585 | F1: 0.0988 | Precision: 0.0
Epoch 28/50 | Train Loss: 0.3043 | Val Loss: 0.7743 | Dice: 0.0914 | IoU: 0.0485 | mAP: 0.0601 | F1: 0.0913 | Precision: 0.0
Epoch 29/50 | Train Loss: 0.3539 | Val Loss: 0.6927 | Dice: 0.1015 | IoU: 0.0541 | mAP: 0.0657 | F1: 0.1015 | Precision: 0.0
Epoch 30/50 | Train Loss: 0.3063 | Val Loss: 0.6672 | Dice: 0.1036 | IoU: 0.0553 | mAP: 0.0664 | F1: 0.1035 | Precision: 0.0
Epoch 31/50 | Train Loss: 0.3106 | Val Loss: 0.7810 | Dice: 0.1297 | IoU: 0.0707 | mAP: 0.0823 | F1: 0.1296 | Precision: 0.0
Checkpoint saved!
Epoch 32/50 | Train Loss: 0.2870 | Val Loss: 1.0682 | Dice: 0.1264 | IoU: 0.0686 | mAP: 0.0774 | F1: 0.1263 | Precision: 0.0
Epoch 33/50 | Train Loss: 0.3089 | Val Loss: 0.9576 | Dice: 0.1164 | IoU: 0.0627 | mAP: 0.0645 | F1: 0.1163 | Precision: 0.0
Epoch 34/50 | Train Loss: 0.2961 | Val Loss: 0.9012 | Dice: 0.1046 | IoU: 0.0560 | mAP: 0.0655 | F1: 0.1045 | Precision: 0.0
Epoch 35/50 | Train Loss: 0.2946 | Val Loss: 0.9775 | Dice: 0.1161 | IoU: 0.0625 | mAP: 0.0719 | F1: 0.1161 | Precision: 0.0
Epoch 36/50 | Train Loss: 0.3189 | Val Loss: 0.6700 | Dice: 0.0966 | IoU: 0.0513 | mAP: 0.0623 | F1: 0.0965 | Precision: 0.0
Epoch 37/50 | Train Loss: 0.3247 | Val Loss: 0.8962 | Dice: 0.1071 | IoU: 0.0573 | mAP: 0.0653 | F1: 0.1070 | Precision: 0.0
Epoch 38/50 | Train Loss: 0.4060 | Val Loss: 0.7436 | Dice: 0.0978 | IoU: 0.0521 | mAP: 0.0643 | F1: 0.0977 | Precision: 0.0
Epoch 39/50 | Train Loss: 0.3397 | Val Loss: 0.6958 | Dice: 0.0873 | IoU: 0.0460 | mAP: 0.0562 | F1: 0.0873 | Precision: 0.0
Epoch 40/50 | Train Loss: 0.3227 | Val Loss: 0.8091 | Dice: 0.1134 | IoU: 0.0611 | mAP: 0.0692 | F1: 0.1133 | Precision: 0.0
Epoch 41/50 | Train Loss: 0.3065 | Val Loss: 1.0400 | Dice: 0.0978 | IoU: 0.0519 | mAP: 0.0607 | F1: 0.0977 | Precision: 0.0
Epoch 42/50 | Train Loss: 0.2950 | Val Loss: 0.9830 | Dice: 0.1063 | IoU: 0.0567 | mAP: 0.0682 | F1: 0.1062 | Precision: 0.0
Epoch 43/50 | Train Loss: 0.2853 | Val Loss: 0.8764 | Dice: 0.1161 | IoU: 0.0625 | mAP: 0.0751 | F1: 0.1160 | Precision: 0.0
Epoch 44/50 | Train Loss: 0.2777 | Val Loss: 1.2950 | Dice: 0.1154 | IoU: 0.0622 | mAP: 0.0634 | F1: 0.1153 | Precision: 0.0
Epoch 45/50 | Train Loss: 0.2940 | Val Loss: 0.7884 | Dice: 0.1160 | IoU: 0.0625 | mAP: 0.0761 | F1: 0.1159 | Precision: 0.0
Epoch 46/50 | Train Loss: 0.2806 | Val Loss: 0.9673 | Dice: 0.1278 | IoU: 0.0694 | mAP: 0.0804 | F1: 0.1277 | Precision: 0.0
Epoch 47/50 | Train Loss: 0.2742 | Val Loss: 1.2205 | Dice: 0.1259 | IoU: 0.0683 | mAP: 0.0777 | F1: 0.1258 | Precision: 0.0
Epoch 48/50 | Train Loss: 0.2907 | Val Loss: 0.6277 | Dice: 0.1131 | IoU: 0.0607 | mAP: 0.0741 | F1: 0.1130 | Precision: 0.0
Epoch 49/50 | Train Loss: 0.3952 | Val Loss: 0.6548 | Dice: 0.0716 | IoU: 0.0374 | mAP: 0.0488 | F1: 0.0715 | Precision: 0.0
Epoch 50/50 | Train Loss: 0.4338 | Val Loss: 0.5391 | Dice: 0.0870 | IoU: 0.0459 | mAP: 0.0581 | F1: 0.0869 | Precision: 0.0

```

```

import matplotlib.pyplot as plt

epochs = list(range(1, num_epochs + 1))

plt.figure(figsize=(18, 12))

# --- Loss Curve ---
plt.subplot(2, 3, 1)
plt.plot(epochs, train_losses, 'o-', color='orange', label='Train')
plt.plot(epochs, val_losses, 'o-', color='blue', label='Validation')
plt.title("Oversampled Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)

```

```

plt.legend()

# --- Dice Curve ---
plt.subplot(2, 3, 2)
plt.plot(epochs, train_dices, 'o-', color='orange', label='Train')
plt.plot(epochs, val_dices, 'o-', color='blue', label='Validation')
plt.title("Oversampled Dice Score")
plt.xlabel("Epoch")
plt.ylabel("Dice")
plt.grid(True)
plt.legend()

# --- IoU Curve ---
plt.subplot(2, 3, 3)
plt.plot(epochs, train_iou, 'o-', color='orange', label='Train')
plt.plot(epochs, val_iou, 'o-', color='blue', label='Validation')
plt.title("Oversampled IoU Score")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

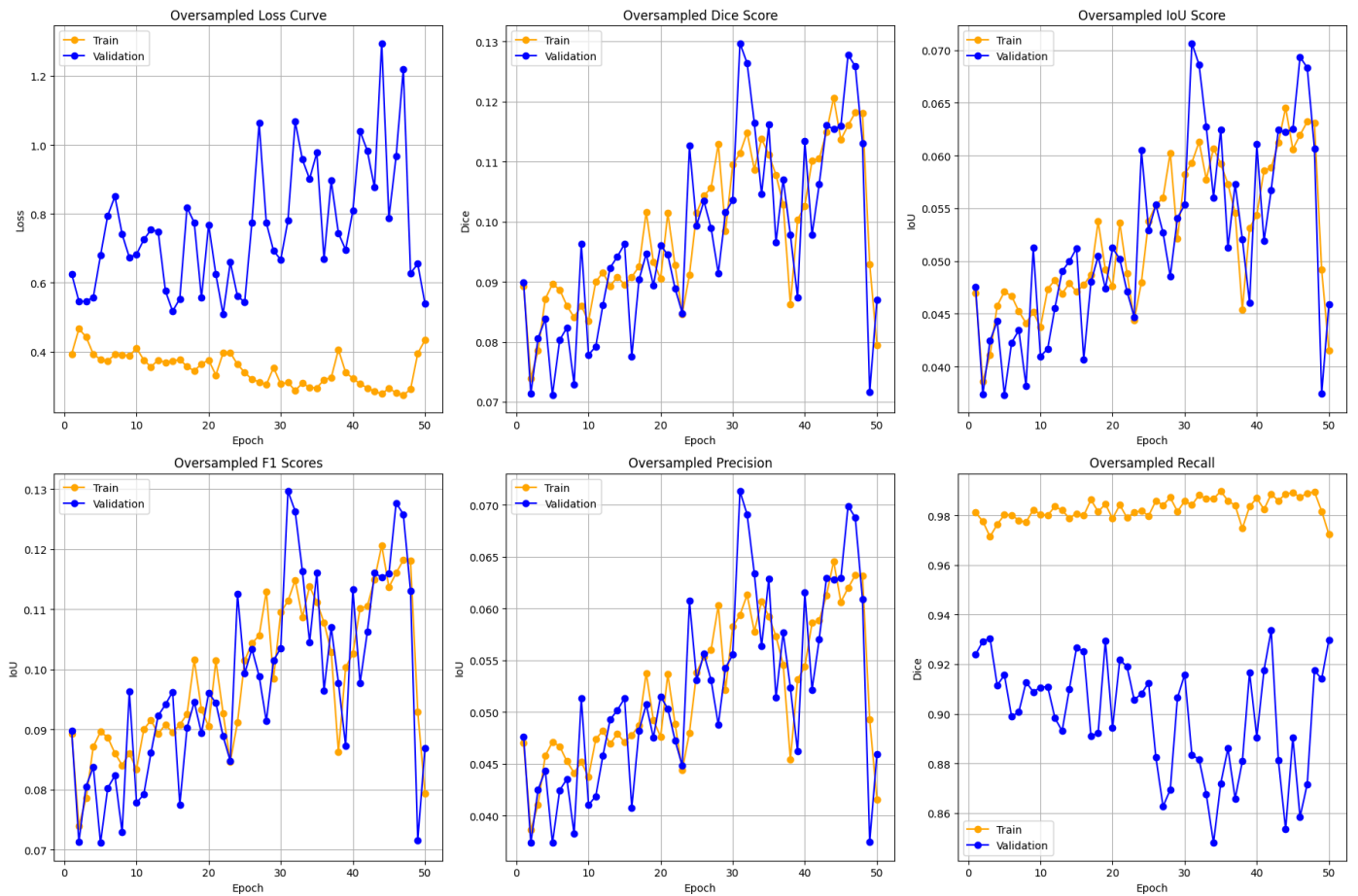
# --- F1 Curve ---
plt.subplot(2, 3, 4)
plt.plot(epochs, train_f1s, 'o-', color='orange', label='Train')
plt.plot(epochs, val_f1s, 'o-', color='blue', label='Validation')
plt.title("Oversampled F1 Scores")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

# --- Precision Curve ---
plt.subplot(2, 3, 5)
plt.plot(epochs, train_precisions, 'o-', color='orange', label='Train')
plt.plot(epochs, val_precisions, 'o-', color='blue', label='Validation')
plt.title("Oversampled Precision")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

# --- Recall Curve ---
plt.subplot(2, 3, 6)
plt.plot(epochs, train_recalls, 'o-', color='orange', label='Train')
plt.plot(epochs, val_recalls, 'o-', color='blue', label='Validation')
plt.title("Oversampled Recall")
plt.xlabel("Epoch")
plt.ylabel("Dice")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import torchvision.transforms.functional as TF

def show_prediction_with_box(model, dataset, index=0, threshold=0.5):
    model.eval()
    with torch.no_grad():
        # Get image and corresponding image_id
        image, _ = dataset[index]
        image_id = dataset.image_ids[index]
        image_info = dataset.coco.loadImgs(image_id)[0]
        filename = image_info['file_name']

        print(f"🖼️ Image ID: {filename}")

        input_tensor = image.unsqueeze(0).to(device)

        # Run model
        output = model(input_tensor)
        output = F.interpolate(output, size=image.shape[1:], mode='bilinear', align_corners=False)
        probs = torch.sigmoid(output)
        pred_mask = (probs > threshold).float().cpu().numpy()[0, 0]

        # Convert to bounding boxes using OpenCV
        mask_uint8 = (pred_mask * 255).astype(np.uint8)
        contours, _ = cv2.findContours(mask_uint8, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        # Original image as NumPy for plotting
```

```
img_np = image.permute(1, 2, 0).cpu().numpy()
img_np = (img_np - img_np.min()) / (img_np.max() - img_np.min()) # Normalize for display
img_disp = (img_np * 255).astype(np.uint8).copy()
```

```
# Draw bounding boxes
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img_disp, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

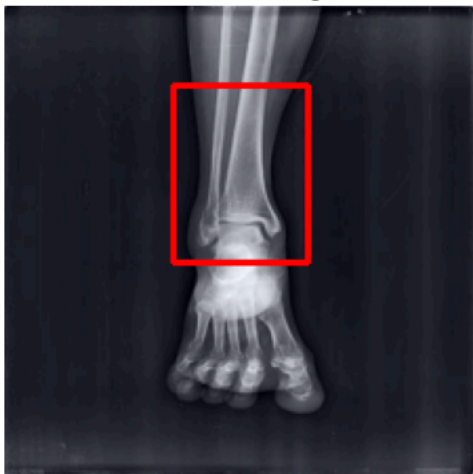
```
# Show image and predicted mask
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(img_disp)
ax[0].set_title("Predicted Bounding Box")
ax[0].axis("off")
```

```
ax[1].imshow(pred_mask, cmap='gray')
ax[1].set_title("Predicted Mask")
ax[1].axis("off")
plt.show()
```

```
# Show bounding box prediction on test image
show_prediction_with_box(model, val_dataset, index=11)
show_prediction_with_box(model, val_dataset, index=10)
```

🔄 🖼️ Image ID: IMG0003803.jpg

Predicted Bounding Box

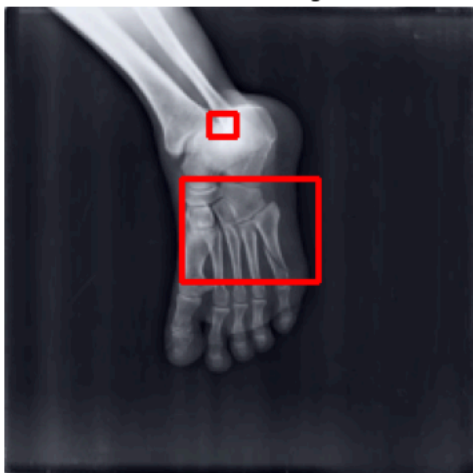


Predicted Mask



🖼️ Image ID: IMG0003798.jpg

Predicted Bounding Box



Predicted Mask



```
import matplotlib.pyplot as plt
import cv2
from PIL import Image
def show_coco_box_from_original_image(dataset, index=0, image_root="/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas

    image_id = dataset.image_ids[index]
    image_info = dataset.coco.loadImgs(image_id)[0]
```

```

filename = image_info['file_name']
filepath = os.path.join(image_root, filename)

print(f"🖼️ Raw image with COCO boxes: {filename}")

# Load original image
img = cv2.imread(filepath)
if img is None:
    print("❌ Failed to load image.")
    return
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Get annotations and draw boxes
ann_ids = dataset.coco.getAnnIds(imgIds=image_id)
anns = dataset.coco.loadAnns(ann_ids)

for i, ann in enumerate(anns):
    x, y, w, h = map(int, ann['bbox']) # COCO box format
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2) # 🟢 Green GT box
    print(f"🟢 GT Box {i+1}: x={x}, y={y}, w={w}, h={h}")

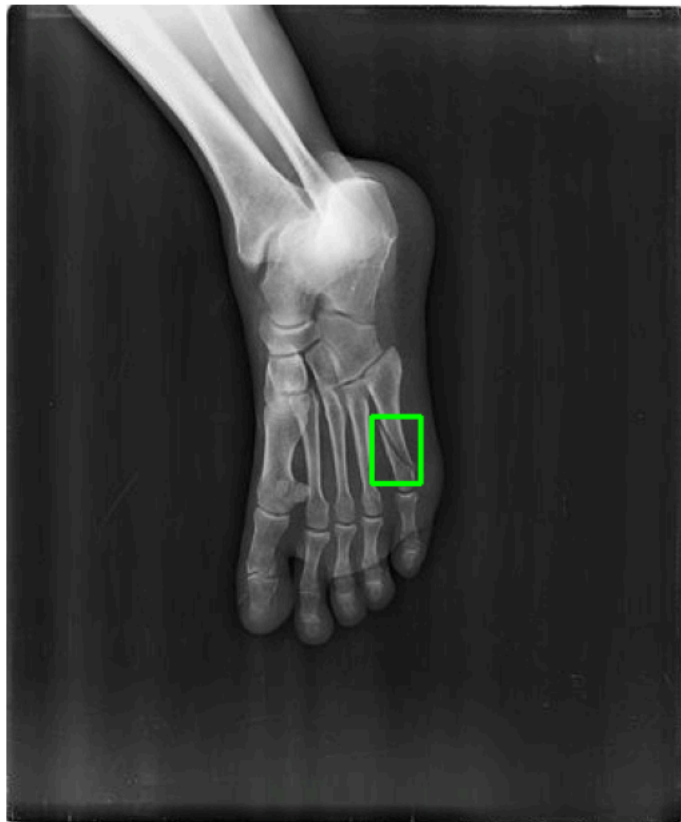
plt.figure(figsize=(8, 8))
plt.imshow(img)
plt.title("🟢 COCO GT Boxes on Original Image")
plt.axis("off")
plt.show()

show_coco_box_from_original_image(val_dataset, index=10)
show_coco_box_from_original_image(val_dataset, index=11)

```

Raw image with COCO boxes: IMG0003798.jpg
GT Box 1: x=201, y=227, w=27, h=37
/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128309 (\N{LARGE BLUE CIRCLE}) missing from font. fig.canvas.print_figure(bytes_io, **kw)

□ COCO GT Boxes on Original Image



Raw image with COCO boxes: IMG0003803.jpg
GT Box 1: x=154, y=191, w=16, h=23

□ COCO GT Boxes on Original Image



```
test_metrics = test_model(model, test_loader, device)
```

```
=== Test Metrics ===  
IoU: 0.0315  
Precision: 0.0317  
Recall: 0.8335  
F1: 0.0604  
mAP: 0.1396  
Dice: 0.0604
```

```
train_csv3 = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/train_combined.csv'  
val_csv3 = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/valid_combined.csv'  
test_csv3 = '/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Utilities/Fracture Split/test_combined.csv'  
ann_path3 = "/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/Annotations/COCO_JSON/COCO_fracture_masks.json"  
images_dir3 = "/content/drive/MyDrive/Health_and_Medicine_Project/FracAtlas/images/sorted_images/Combined/all"
```

```
# Load train/val splits  
train_ids3 = pd.read_csv(train_csv3)['image_id'].tolist()  
val_ids3 = pd.read_csv(val_csv3)['image_id'].tolist()  
test_ids3 = pd.read_csv(test_csv3)['image_id'].tolist()  
  
# Datasets  
train_dataset3 = FracAtlasCocoDataset(images_dir3, ann_path3, train_ids3, transform=transform_train)  
val_dataset3 = FracAtlasCocoDataset(images_dir3, ann_path3, val_ids3, transform=transform_test)  
test_dataset3 = FracAtlasCocoDataset(images_dir3, ann_path3, test_ids3, transform=transform_test)
```

```
# DataLoaders  
train_loader3 = DataLoader(train_dataset3, batch_size=8, shuffle=True)  
val_loader3 = DataLoader(val_dataset3, batch_size=4, shuffle=False)  
test_loader3 = DataLoader(test_dataset3, batch_size=4, shuffle=False)
```

```
loading annotations into memory...  
Done (t=0.01s)  
creating index...  
index created!  
loading annotations into memory...  
Done (t=0.01s)  
creating index...  
index created!  
loading annotations into memory...  
Done (t=0.01s)  
creating index...  
index created!
```

```
# Estimate how imbalanced your dataset is  
total_pixels = 0  
fracture_pixels = 0  
  
for _, mask in train_loader:  
    fracture_pixels += mask.sum().item()  
    total_pixels += mask.numel()  
    break # remove break to compute over the full dataset if you want
```

```
fracture_ratio3 = fracture_pixels / total_pixels  
print(f"Fracture pixel ratio: {fracture_ratio3:.6f}")
```

```
Fracture pixel ratio: 0.002795
```

```
# Compute pos_weight dynamically  
pos_weight_value3 = 1.0 / (fracture_ratio3 + 1e-8)  
print(f"Using pos_weight={pos_weight_value3:.2f}")  
  
# Set loss with pos_weight  
pos_weight3 = torch.tensor([pos_weight_value3]).to(device)  
criterion3 = nn.BCEWithLogitsLoss(pos_weight=pos_weight3)
```

```
Using pos_weight=383.02
```

```
import torch.nn.functional as F
```

```
train_losses3 = []  
train_dices3 = []  
train_iious3 = []  
train_f1s3 = []  
train_precisions3 = []
```



```

train_recalls3 = []
train_accuracies3 = []

val_losses3 = []
val_dices3 = []
val_iou3 = []
val_f1s3 = []
val_precisions3 = []
val_recalls3 = []
val_accuracies3 = []
val_maps3 = []

num_epochs3 = 50
best_val_dice3 = 0.0

for epoch in range(num_epochs3):
    model.train()
    total_loss3 = 0.0
    total_dice3 = 0.0
    total_iou3 = 0.0
    total_f13 = 0.0
    total_precision3 = 0.0
    total_recall3 = 0.0
    total_acc3 = 0.0

    for images3, masks3 in train_loader3:
        images3, masks3 = images3.to(device), masks3.to(device)

        optimizer.zero_grad()
        outputs3 = model(images3)
        outputs3 = F.interpolate(outputs3, size=masks3.shape[2:], mode='bilinear', align_corners=False)
        loss3 = criterion(outputs3, masks3)
        loss3.backward()
        optimizer.step()
        total_loss3 += loss3.item()

        with torch.no_grad():
            probs3 = torch.sigmoid(outputs3)
            preds3 = (probs3 > 0.5).float()
            total_dice3 += dice_score(preds3, masks3).item()
            total_iou3 += iou_score(preds3, masks3).item()
            total_f13 += f1_score(preds3, masks3).item()
            total_precision3 += precision_score(preds3, masks3).item()
            total_recall3 += recall_score(preds3, masks3).item()
            total_acc3 += accuracy_score(preds3, masks3).item()

    conf_matrix3 = {"TP": 0, "TN": 0, "FP": 0, "FN": 0}

    # Average training metrics for this epoch
    avg_train_loss3 = total_loss3 / len(train_loader3)
    avg_train_dice3 = total_dice3 / len(train_loader3)
    avg_train_iou3 = total_iou3 / len(train_loader3)
    avg_train_f13 = total_f13 / len(train_loader3)
    avg_train_precision3 = total_precision3 / len(train_loader3)
    avg_train_recall3 = total_recall3 / len(train_loader3)
    avg_train_acc3 = total_acc3 / len(train_loader3)

    # Store training metrics
    train_losses3.append(avg_train_loss3)
    train_dices3.append(avg_train_dice3)
    train_iou3.append(avg_train_iou3)
    train_f1s3.append(avg_train_f13)
    train_precisions3.append(avg_train_precision3)
    train_recalls3.append(avg_train_recall3)
    train_accuracies3.append(avg_train_acc3)

    # --- VALIDATION ---
    model.eval()
    total_dice3 = 0.0
    total_iou3 = 0.0
    total_f13 = 0.0
    total_precision3 = 0.0
    total_recall3 = 0.0
    total_acc3 = 0.0
    total_loss_val3 = 0.0

    with torch.no_grad():

```

```

all_preds3 = []
all_targets3 = []
for images3, masks3 in val_loader3:
    images3, masks3 = images3.to(device), masks3.to(device)
    outputs3 = model(images3)
    outputs3 = F.interpolate(outputs3, size=masks3.shape[2:], mode='bilinear', align_corners=False)
    loss_val3 = criterion3(outputs3, masks3)
    total_loss_val3 += loss_val.item()
    probs3 = torch.sigmoid(outputs3)
    preds3 = (probs3 > 0.5).float()

    batch_matrix3 = compute_confusion_matrix(preds3, masks3)
    for k in conf_matrix3:
        conf_matrix3[k] += batch_matrix3[k]

    total_dice3 += dice_score(preds3, masks3).item()
    total_iou3 += iou_score(preds3, masks3).item()
    total_f13 += f1_score(preds3, masks3).item()
    total_precision3 += precision_score(preds3, masks3).item()
    total_recall3 += recall_score(preds3, masks3).item()
    total_acc3 += accuracy_score(preds3, masks3).item()
    all_preds3.append(probs3.cpu())
    all_targets3.append(masks3.cpu())

avg_val_loss3 = total_loss_val3 / len(val_loader3)
avg_val_dice3 = total_dice3 / len(val_loader3)
avg_val_iou3 = total_iou3 / len(val_loader3)
avg_val_f13 = total_f13 / len(val_loader3)
avg_val_precision3 = total_precision3 / len(val_loader3)
avg_val_recall3 = total_recall3 / len(val_loader3)
avg_val_acc3 = total_acc3 / len(val_loader3)

val_losses3.append(avg_val_loss3)
val_dices3.append(avg_val_dice3)
val_iou3.append(avg_val_iou3)
val_f1s3.append(avg_val_f13)
val_precisions3.append(avg_val_precision3)
val_recalls3.append(avg_val_recall3)
val_accuracies3.append(avg_val_acc3)

all_preds3 = torch.cat(all_preds3, dim=0)
all_targets3 = torch.cat(all_targets3, dim=0)

val_map3 = compute_mAP(all_preds3, all_targets3)
val_maps3.append(val_map3)

print(f"Epoch {epoch+1}/{num_epochs3} | "
      f"Train Loss: {avg_train_loss3:.4f} | Val Loss: {avg_val_loss3:.4f} | "
      f"Dice: {avg_val_dice3:.4f} | IoU: {avg_val_iou3:.4f} | "
      f"mAP: {val_map3:.4f} | "
      f"F1: {avg_val_f13:.4f} | Precision: {avg_val_precision3:.4f} | Recall: {avg_val_recall3:.4f} | Acc: {avg_val_acc3:.4f}")

if avg_val_dice3 > best_val_dice3:
    best_val_dice3 = avg_val_dice3
    save_checkpoint(model, optimizer, epoch)
    print("Checkpoint saved!")

```

```

➡ Epoch 1/50 | Train Loss: 0.3758 | Val Loss: 0.1375 | Dice: 0.0949 | IoU: 0.0504 | mAP: 0.0581 | F1: 0.0949 | Precision: 0.05
Checkpoint saved!
Epoch 2/50 | Train Loss: 0.3106 | Val Loss: 0.1375 | Dice: 0.0955 | IoU: 0.0508 | mAP: 0.0581 | F1: 0.0954 | Precision: 0.05
Checkpoint saved!
Epoch 3/50 | Train Loss: 0.2880 | Val Loss: 0.1375 | Dice: 0.1172 | IoU: 0.0630 | mAP: 0.0581 | F1: 0.1171 | Precision: 0.06
Checkpoint saved!
Epoch 4/50 | Train Loss: 0.2759 | Val Loss: 0.1375 | Dice: 0.1340 | IoU: 0.0730 | mAP: 0.0581 | F1: 0.1339 | Precision: 0.07
Checkpoint saved!
Epoch 5/50 | Train Loss: 0.2508 | Val Loss: 0.1375 | Dice: 0.1295 | IoU: 0.0702 | mAP: 0.0581 | F1: 0.1294 | Precision: 0.07
Epoch 6/50 | Train Loss: 0.2764 | Val Loss: 0.1375 | Dice: 0.1424 | IoU: 0.0783 | mAP: 0.0581 | F1: 0.1423 | Precision: 0.07
Checkpoint saved!
Epoch 7/50 | Train Loss: 0.2611 | Val Loss: 0.1375 | Dice: 0.1055 | IoU: 0.0564 | mAP: 0.0581 | F1: 0.1054 | Precision: 0.05
Epoch 8/50 | Train Loss: 0.2599 | Val Loss: 0.1375 | Dice: 0.1210 | IoU: 0.0653 | mAP: 0.0581 | F1: 0.1209 | Precision: 0.06
Epoch 9/50 | Train Loss: 0.2825 | Val Loss: 0.1375 | Dice: 0.1239 | IoU: 0.0670 | mAP: 0.0581 | F1: 0.1238 | Precision: 0.06
Epoch 10/50 | Train Loss: 0.2432 | Val Loss: 0.1375 | Dice: 0.1160 | IoU: 0.0624 | mAP: 0.0581 | F1: 0.1159 | Precision: 0.0
Epoch 11/50 | Train Loss: 0.3336 | Val Loss: 0.1375 | Dice: 0.1211 | IoU: 0.0654 | mAP: 0.0581 | F1: 0.1210 | Precision: 0.0
Epoch 12/50 | Train Loss: 0.2465 | Val Loss: 0.1375 | Dice: 0.1214 | IoU: 0.0656 | mAP: 0.0581 | F1: 0.1213 | Precision: 0.0
Epoch 13/50 | Train Loss: 0.2486 | Val Loss: 0.1375 | Dice: 0.1241 | IoU: 0.0671 | mAP: 0.0581 | F1: 0.1240 | Precision: 0.0
Epoch 14/50 | Train Loss: 0.2529 | Val Loss: 0.1375 | Dice: 0.1326 | IoU: 0.0719 | mAP: 0.0581 | F1: 0.1325 | Precision: 0.0
Epoch 15/50 | Train Loss: 0.3398 | Val Loss: 0.1375 | Dice: 0.1081 | IoU: 0.0578 | mAP: 0.0581 | F1: 0.1081 | Precision: 0.0
Epoch 16/50 | Train Loss: 0.2987 | Val Loss: 0.1375 | Dice: 0.1310 | IoU: 0.0709 | mAP: 0.0581 | F1: 0.1309 | Precision: 0.0
Epoch 17/50 | Train Loss: 0.2769 | Val Loss: 0.1375 | Dice: 0.1162 | IoU: 0.0625 | mAP: 0.0581 | F1: 0.1161 | Precision: 0.0

```

Epoch 18/50	Train Loss: 0.2649	Val Loss: 0.1375	Dice: 0.1227	IoU: 0.0661	mAP: 0.0581	F1: 0.1226	Precision: 0.0
Epoch 19/50	Train Loss: 0.2388	Val Loss: 0.1375	Dice: 0.1194	IoU: 0.0644	mAP: 0.0581	F1: 0.1193	Precision: 0.0
Epoch 20/50	Train Loss: 0.2655	Val Loss: 0.1375	Dice: 0.1245	IoU: 0.0674	mAP: 0.0581	F1: 0.1244	Precision: 0.0
Epoch 21/50	Train Loss: 0.2590	Val Loss: 0.1375	Dice: 0.1207	IoU: 0.0649	mAP: 0.0581	F1: 0.1206	Precision: 0.0
Epoch 22/50	Train Loss: 0.2681	Val Loss: 0.1375	Dice: 0.1091	IoU: 0.0584	mAP: 0.0581	F1: 0.1091	Precision: 0.0
Epoch 23/50	Train Loss: 0.3089	Val Loss: 0.1375	Dice: 0.1327	IoU: 0.0722	mAP: 0.0581	F1: 0.1326	Precision: 0.0
Epoch 24/50	Train Loss: 0.2571	Val Loss: 0.1375	Dice: 0.1359	IoU: 0.0740	mAP: 0.0581	F1: 0.1358	Precision: 0.0
Epoch 25/50	Train Loss: 0.2430	Val Loss: 0.1375	Dice: 0.1244	IoU: 0.0673	mAP: 0.0581	F1: 0.1243	Precision: 0.0
Epoch 26/50	Train Loss: 0.2367	Val Loss: 0.1375	Dice: 0.1348	IoU: 0.0733	mAP: 0.0581	F1: 0.1347	Precision: 0.0
Epoch 27/50	Train Loss: 0.2339	Val Loss: 0.1375	Dice: 0.1406	IoU: 0.0767	mAP: 0.0581	F1: 0.1405	Precision: 0.0
Epoch 28/50	Train Loss: 0.2098	Val Loss: 0.1375	Dice: 0.1686	IoU: 0.0939	mAP: 0.0581	F1: 0.1685	Precision: 0.0
Checkpoint saved!							
Epoch 29/50	Train Loss: 0.2295	Val Loss: 0.1375	Dice: 0.1545	IoU: 0.0854	mAP: 0.0581	F1: 0.1544	Precision: 0.0
Epoch 30/50	Train Loss: 0.2295	Val Loss: 0.1375	Dice: 0.1429	IoU: 0.0787	mAP: 0.0581	F1: 0.1428	Precision: 0.0
Epoch 31/50	Train Loss: 0.2128	Val Loss: 0.1375	Dice: 0.1453	IoU: 0.0799	mAP: 0.0581	F1: 0.1452	Precision: 0.0
Epoch 32/50	Train Loss: 0.2041	Val Loss: 0.1375	Dice: 0.1544	IoU: 0.0858	mAP: 0.0581	F1: 0.1543	Precision: 0.0
Epoch 33/50	Train Loss: 0.2063	Val Loss: 0.1375	Dice: 0.1496	IoU: 0.0823	mAP: 0.0581	F1: 0.1495	Precision: 0.0
Epoch 34/50	Train Loss: 0.2452	Val Loss: 0.1375	Dice: 0.1025	IoU: 0.0547	mAP: 0.0581	F1: 0.1024	Precision: 0.0
Epoch 35/50	Train Loss: 0.2739	Val Loss: 0.1375	Dice: 0.1107	IoU: 0.0593	mAP: 0.0581	F1: 0.1106	Precision: 0.0
Epoch 36/50	Train Loss: 0.2895	Val Loss: 0.1375	Dice: 0.1186	IoU: 0.0640	mAP: 0.0581	F1: 0.1185	Precision: 0.0
Epoch 37/50	Train Loss: 0.2407	Val Loss: 0.1375	Dice: 0.1293	IoU: 0.0702	mAP: 0.0581	F1: 0.1293	Precision: 0.0
Epoch 38/50	Train Loss: 0.2459	Val Loss: 0.1375	Dice: 0.1528	IoU: 0.0846	mAP: 0.0581	F1: 0.1527	Precision: 0.0
Epoch 39/50	Train Loss: 0.2475	Val Loss: 0.1375	Dice: 0.1098	IoU: 0.0591	mAP: 0.0581	F1: 0.1098	Precision: 0.0
Epoch 40/50	Train Loss: 0.2171	Val Loss: 0.1375	Dice: 0.1216	IoU: 0.0661	mAP: 0.0581	F1: 0.1215	Precision: 0.0
Epoch 41/50	Train Loss: 0.2333	Val Loss: 0.1375	Dice: 0.1386	IoU: 0.0761	mAP: 0.0581	F1: 0.1385	Precision: 0.0
Epoch 42/50	Train Loss: 0.2460	Val Loss: 0.1375	Dice: 0.1374	IoU: 0.0751	mAP: 0.0581	F1: 0.1373	Precision: 0.0
Epoch 43/50	Train Loss: 0.2192	Val Loss: 0.1375	Dice: 0.1578	IoU: 0.0874	mAP: 0.0581	F1: 0.1577	Precision: 0.0
Epoch 44/50	Train Loss: 0.2397	Val Loss: 0.1375	Dice: 0.1309	IoU: 0.0711	mAP: 0.0581	F1: 0.1308	Precision: 0.0
Epoch 45/50	Train Loss: 0.2196	Val Loss: 0.1375	Dice: 0.1368	IoU: 0.0746	mAP: 0.0581	F1: 0.1367	Precision: 0.0
Epoch 46/50	Train Loss: 0.2027	Val Loss: 0.1375	Dice: 0.1365	IoU: 0.0744	mAP: 0.0581	F1: 0.1364	Precision: 0.0
Epoch 47/50	Train Loss: 0.2051	Val Loss: 0.1375	Dice: 0.1625	IoU: 0.0899	mAP: 0.0581	F1: 0.1624	Precision: 0.0
Epoch 48/50	Train Loss: 0.2049	Val Loss: 0.1375	Dice: 0.1402	IoU: 0.0767	mAP: 0.0581	F1: 0.1401	Precision: 0.0
Epoch 49/50	Train Loss: 0.2082	Val Loss: 0.1375	Dice: 0.1516	IoU: 0.0834	mAP: 0.0581	F1: 0.1515	Precision: 0.0
Epoch 50/50	Train Loss: 0.2164	Val Loss: 0.1375	Dice: 0.1444	IoU: 0.0792	mAP: 0.0581	F1: 0.1443	Precision: 0.0

```

import matplotlib.pyplot as plt

epochs = list(range(1, num_epochs + 1))

plt.figure(figsize=(18, 12))

# --- Loss Curve ---
plt.subplot(2, 3, 1)
plt.plot(epochs, train_losses3, 'o-', color='orange', label='Train')
plt.plot(epochs, val_losses3, 'o-', color='blue', label='Validation')
plt.title("Higher Threshold Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.legend()

# --- Dice Curve ---
plt.subplot(2, 3, 2)
plt.plot(epochs, train_dices3, 'o-', color='orange', label='Train')
plt.plot(epochs, val_dices3, 'o-', color='blue', label='Validation')
plt.title("Higher Threshold Dice Score")
plt.xlabel("Epoch")
plt.ylabel("Dice")
plt.grid(True)
plt.legend()

# --- IoU Curve ---
plt.subplot(2, 3, 3)
plt.plot(epochs, train_ious3, 'o-', color='orange', label='Train')
plt.plot(epochs, val_ious3, 'o-', color='blue', label='Validation')
plt.title("Higher Threshold IoU Score")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

# --- F1 Curve ---
plt.subplot(2, 3, 4)
plt.plot(epochs, train_f1s3, 'o-', color='orange', label='Train')
plt.plot(epochs, val_f1s3, 'o-', color='blue', label='Validation')
plt.title("Higher Threshold F1 Scores")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)

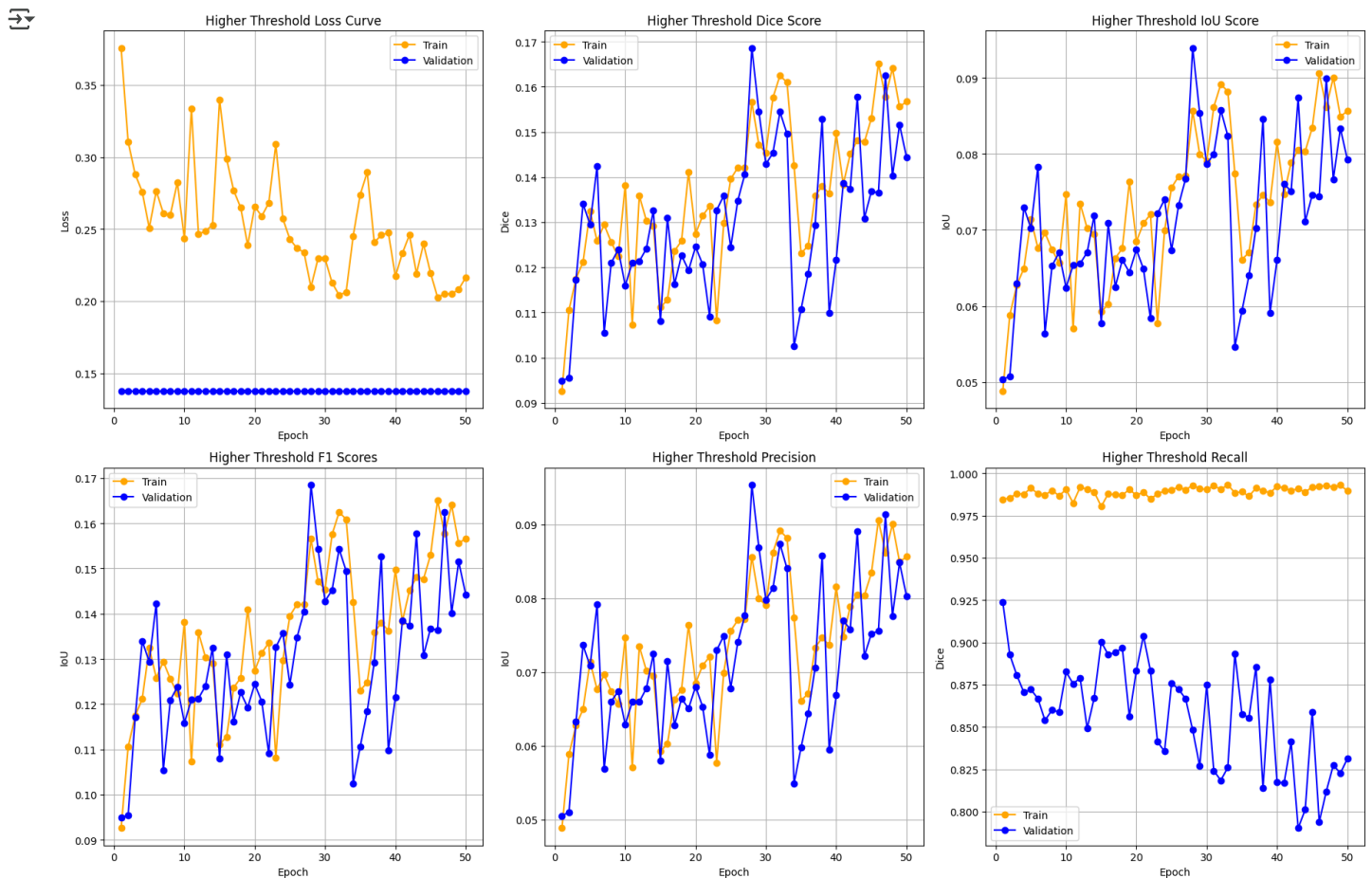
```

```
plt.legend()

# --- Precision Curve ---
plt.subplot(2, 3, 5)
plt.plot(epochs, train_precisions3, 'o-', color='orange', label='Train')
plt.plot(epochs, val_precisions3, 'o-', color='blue', label='Validation')
plt.title("Higher Threshold Precision")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.grid(True)
plt.legend()

# --- Recall Curve ---
plt.subplot(2, 3, 6)
plt.plot(epochs, train_recalls3, 'o-', color='orange', label='Train')
plt.plot(epochs, val_recalls3, 'o-', color='blue', label='Validation')
plt.title("Higher Threshold Recall")
plt.xlabel("Epoch")
plt.ylabel("Dice")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import torchvision.transforms.functional as TF
```